

IMDB SQL Practice — Basic to Advanced

Complete table structures first, followed by 100 SQL practice problems

Part 1 — IMDB Database Table Structures

The following structures are based directly on the uploaded IMDB MySQL dump.

TABLE 1: actors

Attribute	Data Type	NULL / Default	Key / Index
id	int(11)	NOT NULL DEFAULT '0'	PRIMARY KEY
first_name	varchar(100)	DEFAULT NULL	—
last_name	varchar(100)	DEFAULT NULL	—
gender	char(1)	DEFAULT NULL	—
film_count	int(11)	DEFAULT '0'	—

TABLE 2: actors2

Attribute	Data Type	NULL / Default	Key / Index
id	int(11)	DEFAULT NULL	—
film_count	int(11)	DEFAULT '0'	—

TABLE 3: directors

Attribute	Data Type	NULL / Default	Key / Index
id	int(11)	NOT NULL DEFAULT '0'	PRIMARY KEY
first_name	varchar(100)	DEFAULT NULL	—
last_name	varchar(100)	DEFAULT NULL	—

TABLE 4: directors_genres

Attribute	Data Type	NULL / Default	Key / Index
director_id	int(11)	DEFAULT NULL	INDEX: idx_director_id
genre	varchar(100)	DEFAULT NULL	—
prob	float	DEFAULT NULL	—

TABLE 5: movies

Attribute	Data Type	NULL / Default	Key / Index
id	int(11)	NOT NULL DEFAULT '0'	PRIMARY KEY
name	varchar(100)	DEFAULT NULL	—
year	int(11)	DEFAULT NULL	—
rank	float	DEFAULT NULL	—

TABLE 6: movies_directors

Attribute	Data Type	NULL / Default	Key / Index
director_id	int(11)	DEFAULT NULL	INDEX: idx_director_id

Attribute	Data Type	NULL / Default	Key / Index
movie_id	int(11)	DEFAULT NULL	INDEX: idx_movie_id

TABLE 7: movies_genres

Attribute	Data Type	NULL / Default	Key / Index
movie_id	int(11)	DEFAULT NULL	—
genre	varchar(100)	DEFAULT NULL	—

TABLE 8: roles

Attribute	Data Type	NULL / Default	Key / Index
actor_id	int(11)	DEFAULT NULL	INDEX: idx_actor_id
movie_id	int(11)	DEFAULT NULL	INDEX: idx_movie_id
role	varchar(100)	DEFAULT NULL	—

Logical Join Relationships

- actors.id ↔ roles.actor_id
- movies.id ↔ roles.movie_id
- movies.id ↔ movies_genres.movie_id
- movies.id ↔ movies_directors.movie_id
- directors.id ↔ movies_directors.director_id
- directors.id ↔ directors_genres.director_id

Part 2 — SQL Practice Questions

Try to solve each question yourself first. Hints are provided from the medium-level JOIN problems onward.

Level 1 — Basic SELECT & Filtering

1. **Display all movies** — Show all columns from the movies table.
2. **Display movie names** — Show only the name column from movies.
3. **Movies after 2000** — Find all movies released after the year 2000.
4. **Movies from 1990** — Find all movies released in 1990.
5. **High-rated movies** — Find movies having a rank greater than 8.
6. **Low-rated movies** — Find movies with a rank less than 5.
7. **Male actors** — Display all male actors.
8. **Female actors** — Display all female actors.
9. **Actors with more than 5 films** — Find actors whose film_count is greater than 5.
10. **Movies with missing ratings** — Find movies where rank is NULL.

Level 2 — ORDER BY, LIMIT & Aggregation

11. **Top 10 highest-rated movies** — Return the 10 movies with the highest rank.
12. **Oldest movies** — Find the 10 oldest movies in the database.
13. **Most prolific actors** — Find the top 10 actors according to film_count.
14. **Number of movies** — Find the total number of movies.
15. **Number of actors** — Find the total number of actors.
16. **Average movie rating** — Calculate the average rank of all movies.
17. **Highest movie rating** — Find the highest movie rank.
18. **Lowest movie rating** — Find the lowest movie rank.
19. **Movies per year** — Find how many movies were released in each year.
20. **Average rating by year** — Calculate the average movie rating for every year.

Level 3 — GROUP BY & HAVING

21. **Years with more than 100 movies** — Find years containing more than 100 movies.
22. **Number of movies by genre** — Use movies_genres to find the number of movies in each genre.
23. **Average rating by genre** — Find the average movie rating for each genre.
24. **Genres with more than 500 movies** — Return only genres having more than 500 movies.
25. **Number of movies directed by each director** — Use movies_directors.
26. **Directors with more than 5 movies** — Find directors who have directed more than 5 movies.
27. **Number of movies for each actor** — Use roles to calculate how many movies each actor appears in.
28. **Actors appearing in more than 10 movies** — Use roles and GROUP BY.
29. **Number of actors by gender** — Find the number of male and female actors.
30. **Average genre probability by director** — Use directors_genres and calculate average prob for each director.

Level 4 — INNER JOIN

31. **Movie name + genre** — Display movie_name and genre using movies and movies_genres.

Hint: Use movie_id as the link between movies_genres and movies.

32. **Movie name + director ID** — Join movies and movies_directors.

Hint: movies_directors connects movie_id to director_id.

33. **Movie name + director name** — Join movies, movies_directors and directors.

Hint: Join movies → movies_directors → directors.

34. **Actor + movie** — Join actors, roles and movies; show actor, movie and role.

Hint: roles connects actor_id and movie_id.

35. Find movies starring Kevin Bacon — Find all movies in which Kevin Bacon appears.

Hint: Find Kevin Bacon's actor ID first.

36. Find directors named Christopher — Find directors whose first name contains Christopher.

Hint: Use LIKE with % on first_name.

37. Movies directed by a particular director — Pick a director and display all movies directed by them.

Hint: Filter by director_id after joining.

38. Actors and their movie count — Compare actors.film_count with the number of movies obtained from roles.

Hint: COUNT(DISTINCT movie_id) per actor.

39. Movie + genre + rating — Display movie_name, genre and rank.

Hint: Join movies_genres to movies.

40. Movie + director + rating — Display movie_name, director name and rank.

Hint: Join movies → movies_directors → directors.

Level 5 — Multi-Table JOINS

41. Actors in movies directed by a specific director — Use actors, roles, movies, movies_directors and directors.

Hint: Build actors → roles → movies → movies_directors → directors.

42. Movies starring Kevin Bacon with ratings — Return movie_name, year and rank.

Hint: Filter Kevin Bacon, then join roles to movies.

43. Actors who worked with Kevin Bacon — Find actors who appeared in at least one movie with Kevin Bacon; exclude Kevin Bacon.

Hint: Self-join roles on movie_id using two actor aliases.

44. Directors who worked with Kevin Bacon — Return directors who directed a movie featuring Kevin Bacon.

Hint: Find Kevin Bacon's movie IDs, then directors.

45. Actors who worked with a particular director — Identify every actor appearing in that director's movies.

Hint: Get director movie IDs, then roles.

46. Movies having multiple genres — Find movies that belong to more than one genre.

Hint: GROUP BY movie_id + HAVING COUNT(DISTINCT genre) > 1.

47. Movies having multiple directors — Find movies associated with more than one director.

Hint: GROUP BY movie_id + HAVING COUNT(DISTINCT director_id) > 1.

48. Directors associated with multiple genres — Use directors_genres.

Hint: GROUP BY director_id and count distinct genres.

49. Actors who worked across multiple genres — Join actors, roles, movies_genres and count unique genres.

Hint: Count DISTINCT genre per actor.

50. Most common actor role — Determine which role name appears most frequently.

Hint: GROUP BY role, ORDER BY COUNT(*) DESC, LIMIT 1.

Level 6 — Subqueries

51. Movies rated above average — Find movies whose rank is greater than the overall average rank.

Hint: Use AVG(rank) in a subquery.

52. Actors with above-average film count — Find actors whose film_count is greater than the average.

Hint: Compare film_count with a subquery containing AVG.

53. Highest-rated movie — Find the movie(s) having the maximum rank.

Hint: Compare rank with MAX(rank).

54. Second-highest-rated movie — Find the second-highest unique movie rating.

Hint: Use DISTINCT ratings plus nested MAX.

55. Directors with above-average movie count — Find directors who directed more movies than the average director.

Hint: Aggregate movie counts per director, then average those counts.

56. Actors in the highest-rated movie — Find all actors appearing in the movie with the highest rank.

Hint: Find highest-rated movie first, then roles.

57. Directors of the highest-rated movie — Find the director(s) of the highest-rated movie.

Hint: Use highest-rated movie ID with movies_directors.

58. Movies rated higher than their year's average — Compare each movie with the average rating for its year.

Hint: Use a correlated subquery by year.

59. Actors with more movies than Kevin Bacon — Compare each actor's appearance count with Kevin Bacon's.

Hint: Compare actor counts with Kevin Bacon's count.

60. Movies with the same rating as the highest-rated movie before 2000 — Use a subquery.

Hint: Find the pre-2000 target rating in a subquery.

Level 7 — Window Functions

61. Top 5 movies for every year — Find the 5 highest-rated movies from each year.

Hint: Use ROW_NUMBER/DENSE_RANK PARTITION BY year.

62. Rank movies within each year — Output movie_name, year, rank and yearly_rank.

Hint: Use a window function with PARTITION BY year.

63. Rank actors by number of movies — Generate actor_rank based on film count.

Hint: RANK() after grouping actors.

64. Top 3 actors for each gender — Find the three actors with the highest film count for each gender.

Hint: Partition the window by gender.

65. Most popular genre for every year — Determine the genre with the most movies in each year.

Hint: Rank genres within each year.

66. Highest-rated movie for every genre — Return genre, movie_name and rank.

Hint: Rank movies within each genre and keep rank 1.

67. Second-highest-rated movie for every genre — Use DENSE_RANK or ROW_NUMBER.

Hint: DENSE_RANK handles ties.

68. Director with the highest-rated movie — Find the director associated with the highest-rated movie.

Hint: Find max movie rank and join to directors.

69. Directors' average movie rating — Calculate movie_count and average_rating, then rank directors.

Hint: AVG(rank) per director, then RANK().

70. Directors with at least 5 movies and average rating > 7 — Combine JOIN, GROUP BY, HAVING and AVG.

Hint: HAVING COUNT(DISTINCT movie_id) >= 5 AND AVG(rank) > 7.

Level 8 — Advanced Analytical SQL

71. Actor collaboration count — For every pair of actors, calculate how many movies they appeared in together.

Hint: Self-join roles on movie_id; enforce actor_id < actor_id.

72. Actor with the most collaborators — Determine which actor has worked with the largest number of different actors.

Hint: Count distinct collaborators per actor.

73. Director with the largest cast — For every director, calculate the number of unique actors in their movies.

Hint: COUNT(DISTINCT actor_id) per director.

74. Actor's average movie rating — For every actor calculate number_of_movies and average_movie_rating.

Hint: AVG(movie rank) grouped by actor.

75. Actors who worked with at least 3 directors — Count distinct directors per actor.

Hint: COUNT(DISTINCT director_id) with HAVING >= 3.

76. Actors who worked across at least 5 genres — Count distinct genres per actor.

Hint: COUNT(DISTINCT genre) with HAVING >= 5.

77. Director genre specialization — For every director, find the genre with the highest probability (prob).

Hint: Rank genres within each director by prob DESC.

78. Most successful genre — Define success as average movie rank and find the best genre.

Hint: GROUP BY genre and AVG(rank).

79. Best actor by genre — For each genre, find the actor with the highest average movie rating.

Hint: Group by genre + actor, then rank within genre.

80. Best director by genre — For every genre, find the director whose movies have the highest average rating.

Hint: Group by genre + director, then rank within genre.

Level 9 — Expert Challenges

81. Actor career span — For every actor find first_movie_year, last_movie_year and career_span.

Hint: MIN(year) and MAX(year) after joining roles to movies.

82. Most active decade — Find which decade produced the most movies.

Hint: Convert year to decade, e.g. FLOOR(year/10)*10.

83. Best decade by average rating — Calculate average movie rating for every decade and find the best.

Hint: GROUP BY decade and AVG(rank).

84. Director career span — For every director find first_movie_year, last_movie_year and career_span.

Hint: MIN/MAX(year) per director.

85. Actor's highest-rated movie — For every actor find their highest-rated movie.

Hint: Rank each actor's movies by rank DESC.

86. Actor's second-best movie — Find the second-highest-rated movie for every actor.

Hint: Use ROW_NUMBER/DENSE_RANK partitioned by actor.

87. Directors with declining ratings — Find directors whose later movies have a lower average rating than their earlier movies.

Hint: Split earlier/later movies and compare averages.

88. Most versatile actors — Find actors who have worked across the greatest number of unique genres.

Hint: COUNT(DISTINCT genre) per actor.

89. Most versatile directors — Find directors associated with the greatest number of genres.

Hint: Count distinct genres per director.

90. Actor network — Build actor_A, actor_B, movies_together and find the most connected actor.

Hint: Build collaboration pairs, count collaborators, then rank.

Level 10 — Interview / Project Challenges

91. Top 10 actors by weighted movie rating — Rank actors using SUM(movie_rank), not just movie count.

Hint: SUM(rank) per actor.

92. Actor quality score — Define quality_score = average_movie_rating × number_of_movies and rank actors.

Hint: AVG(rank) × COUNT(DISTINCT movie_id).

93. Director quality score — Define quality_score = average_rating × movie_count and rank directors.

Hint: AVG(rank) × movie_count per director.

94. Genre performance over time — For every genre and year calculate movie_count and average_rating; identify improving genres.

Hint: Group by year + genre and compare trends.

95. Actors who only worked in highly-rated movies — Define highly-rated as rank ≥ 8 and find actors with no movie below 8.

Hint: Use `MIN(rank) >= 8` while handling NULLs.

96. Directors whose every movie is rated above 7 — Solve using `GROUP BY` and `HAVING`.

Hint: `GROUP BY` director and `MIN(rank) > 7`.

97. Actors who worked with both Director A and Director B — Use `JOIN`/subqueries or an `INTERSECT`-style approach.

Hint: Get both actor sets and intersect them.

98. Strongest actor-director partnership — Calculate how many movies each actor-director pair worked on together.

Hint: `GROUP BY` actor_id + director_id and `COUNT(DISTINCT movie_id)`.

99. IMDB superstar — Create a metric combining number of movies, average rating, number of genres and number of directors.

Hint: Create separate actor metrics and combine them.

100. Complete IMDB analysis — Produce total movies/actors/directors, average rating, top movies/actors/directors, genre/year trends, versatility and collaboration analysis.

Hint: Use separate CTEs for each analysis component.